An introduction to Python Programming for Research

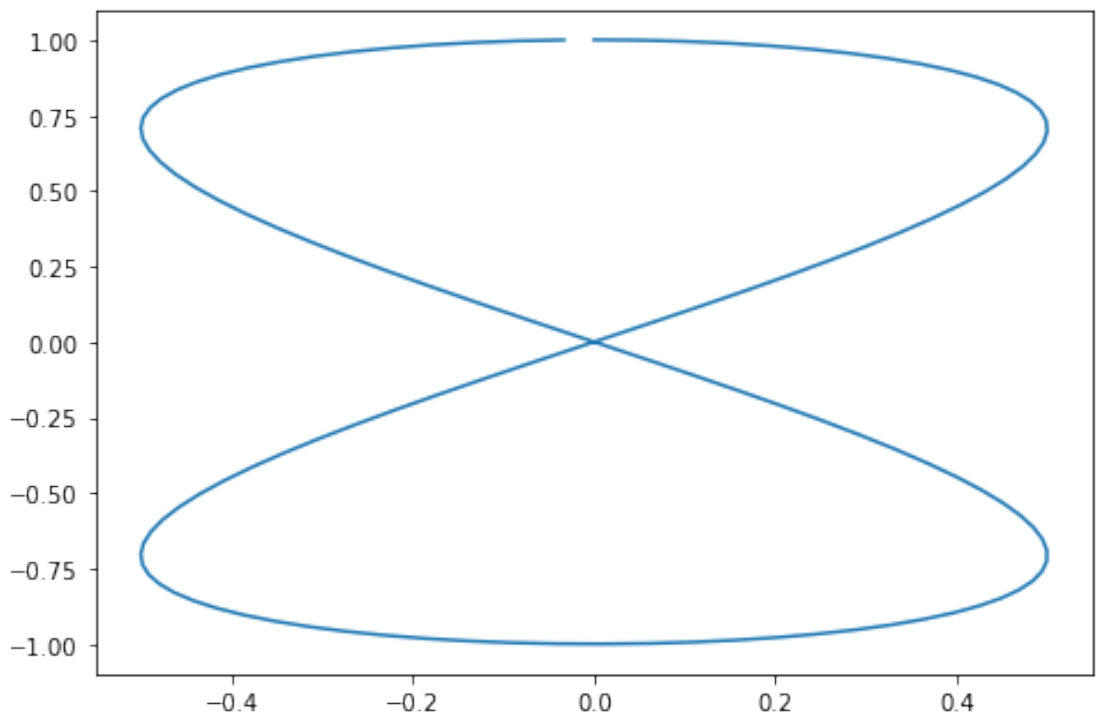
James Hetherington

November 24, 2017

Contents

```
In [1]: ### Make plot
        %matplotlib inline
        import numpy as np
        import math
        import matplotlib.pyplot as plt

        theta=np.arange(0,4*math.pi,0.1)
        eight=plt.figure()
        axes=eight.add_axes([0,0,1,1])
        axes.plot(0.5*np.sin(theta),np.cos(theta/2))
```

```
Out[1]: [<matplotlib.lines.Line2D at 0x2b9b564f3630>]
```




```
In [2]: print("This cell is a code cell")
```

```
This cell is a code cell
```

```
In [3]: %%bash
        # Above line tells Python to execute this cell as *shell code*
        # not Python, as if we were in a command line
        # This is called a 'cell magic'

        python -c "print(2*4)"
```

```
8
```

```
In [4]: %%writefile eight.py
        print(2*4)
```

```
Writing eight.py
```

```
In [5]: %%bash
        python eight.py
```

```
8
```

```
In [6]: %%writefile eight.py
        #! /usr/bin/env python
        print(2*7)
```

```
Overwriting eight.py
```

```
In [7]: %%bash
        chmod u+x eight.py
```

```
In [8]: %%bash
        ./eight.py
```

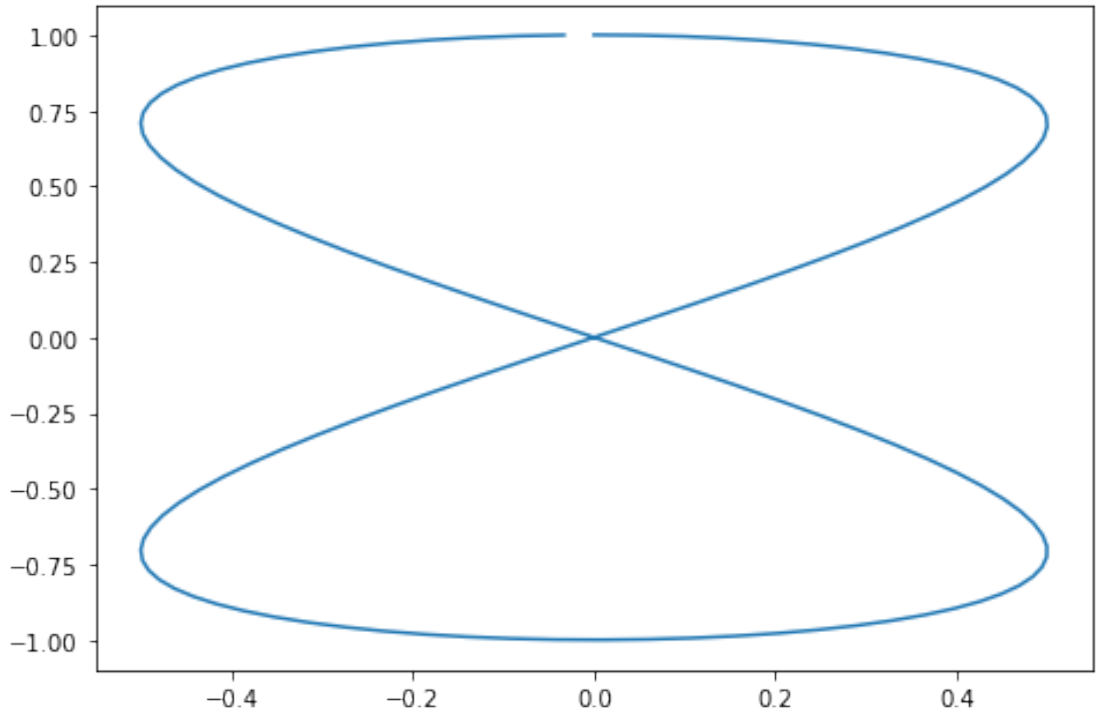
```
14
```

```
In [9]: %%writefile draw_eight.py
        # Above line tells the notebook to treat the rest of this
        # cell as content for a file on disk.

        import numpy as np
        import math
        import matplotlib.pyplot as plt

        def make_figure():
          theta = np.arange(0, 4 * math.pi, 0.1)
          eight = plt.figure()
          axes = eight.add_axes([0, 0, 1, 1])
          axes.plot(0.5 * np.sin(theta), np.cos(theta / 2))
          return eight
```

```
Writing draw_eight.py
```

```
In [10]: import draw_eight # Load the library file we just wrote to disk
```

```
In [11]: image=draw_eight.make_figure()
```




```
In [1]: %matplotlib inline

In [2]: # A python library for investigating geographic information.
        # https://pypi.python.org/pypi/geopy~
        import geopy

In [3]: geocoder=geopy.geocoders.GoogleV3(domain="maps.google.co.uk")
        geocoder.geocode('Cambridge', exactly_one=False)

Out[3]: [Location(Cambridge, UK, (52.205337, 0.121817, 0.0))]

In [4]: print("This runs")# print("This doesn't")
        # print("This doesn't either")

This runs


In [5]: def geolocate(place):
          return geocoder.geocode(place, exactly_one = False)[0][1]

In [6]: geolocate('London')


        ---------------------------------------------------------------------------

        GeocoderQuotaExceeded                     Traceback (most recent call last)
```

```
    <ipython-input-6-734a234399dd> in <module>()
----> 1 geolocate('London')


    <ipython-input-5-2394ca7f2ca5> in geolocate(place)
      1 def geolocate(place):
----> 2   return geocoder.geocode(place, exactly_one = False)[0][1]


    ~/virtualenv/python3.6.3/lib/python3.6/site-packages/geopy/geocoders/googlev3.py in geocode(sel
    215         logger.debug("%s.geocode: %s", self.__class__.__name__, url)
    216         return self._parse_json(
--> 217             self._call_geocoder(url, timeout=timeout), exactly_one
    218         )
    219


    ~/virtualenv/python3.6.3/lib/python3.6/site-packages/geopy/geocoders/googlev3.py in _parse_json
    336         places = page.get('results', [])
    337         if not len(places):
--> 338             self._check_status(page.get('status'))
    339             return None
    340


    ~/virtualenv/python3.6.3/lib/python3.6/site-packages/geopy/geocoders/googlev3.py in _check_stat
    361         if status == 'OVER_QUERY_LIMIT':
    362             raise GeocoderQuotaExceeded(
--> 363                 'The given key has gone over the requests limit in the 24'
    364                 ' hour period or has submitted too many requests in too'
    365                 ' short a period of time.'


    GeocoderQuotaExceeded: The given key has gone over the requests limit in the 24 hour period or
```

```
In [7]: london_location = geolocate("London")
        print(london_location)
```

```
(51.5073509, -0.1277583)
```

```
In [8]: import requests
        def request_map_at(lat, long, satellite=True, zoom=10, size=(400, 400), sensor=False):
            base="http://maps.googleapis.com/maps/api/staticmap?"
            params={
                "sensor": str(sensor).lower(),
                "zoom": zoom,
                "size": "x".join(map(str, size)),
                "center" : ",".join(map(str, (lat, long))),
                "style" : "feature:all|element:labels|visibility:off"
            }
            if satellite:
                params["maptype"] = "satellite"

            return requests.get(base, params=params)
```

```
In [9]: map_response = request_map_at(51.5072, -0.1275)

In [10]: map_response

Out[10]: <Response [200]>

In [11]: url = map_response.url
         print(url[0:50])
         print(url[50:100])
         print(url[100:])
```

```
http://maps.googleapis.com/maps/api/staticmap?sens
or=false&zoom=10&size=400x400&center=51.5072%2C-0.
1275&style=feature%3Aall%7Celement%3Alabels%7Cvisibility%3Aoff&maptype=satellite
```

```
In [12]: assert "http://maps.googleapis.com/maps/api/staticmap?" in url
         assert "center=51.5072%2C-0.1275" in url
         assert "zoom=10" in url
         assert "size=400x400" in url
         assert "sensor=false" in url

In [13]: map_response.content[0:20]

Out[13]: b'\x89PNG\r\n\x1a\n\x00\x00\x00\rIHDR\x00\x00\x01\x90'

In [14]: def map_at(*args, **kwargs):
             return request_map_at(*args, **kwargs).content

In [15]: map_png = map_at(*london_location)

In [16]: print("The type of our map result is actually a: ", type(map_png))
```

```
The type of our map result is actually a:  <class 'bytes'>
```

```
In [17]: # TODO: check the IPYTHON/jUPYTER CORE
         from IPython import display
         display.Image(map_png)

   Out[17]:
```

```
In [18]: from io import BytesIO # An object to convert between files and unicode strings
         import numpy as np # A library to deal with matrices
         from matplotlib import image as img # A library to deal with images
```

```
In [19]: def is_green(pixels):
             threshold=1.1
             greener_than_red = pixels[:,:,1] > threshold * pixels[:,:,0]
             greener_than_blue = pixels[:,:,1] > threshold * pixels[:,:,2]
             green = np.logical_and(greener_than_red, greener_than_blue)
             return green
```

```
In [20]: def count_green_in_png(data):
             pixels = img.imread(BytesIO(data)) # Get our PNG image as a numpy array
             return np.sum(is_green(pixels))
```

```
In [21]: print(count_green_in_png(map_at(*london_location)))
```

```
107222
```

```
In [22]: def location_sequence(start, end, steps):
             lats = np.linspace(start[0], end[0], steps) # "Linearly spaced" data
             longs = np.linspace(start[1],end[1], steps)
             return np.vstack([lats, longs]).transpose()
```

```
In [23]: location_sequence(geolocate("London"), geolocate("Cambridge"), 5)
```

```
---------------------------------------------------------------------------

GeocoderQuotaExceeded                     Traceback (most recent call last)

<ipython-input-23-ed53afe2376e> in <module>()
----> 1 location_sequence(geolocate("London"), geolocate("Cambridge"), 5)


<ipython-input-5-2394ca7f2ca5> in geolocate(place)
      1 def geolocate(place):
----> 2   return geocoder.geocode(place, exactly_one = False)[0][1]


~/virtualenv/python3.6.3/lib/python3.6/site-packages/geopy/geocoders/googlev3.py in geocode(sel
215         logger.debug("%s.geocode: %s", self.__class__.__name__, url)
216         return self._parse_json(
--> 217             self._call_geocoder(url, timeout=timeout), exactly_one
218         )
219


~/virtualenv/python3.6.3/lib/python3.6/site-packages/geopy/geocoders/googlev3.py in _parse_json
336         places = page.get('results', [])
337         if not len(places):
--> 338             self._check_status(page.get('status'))
339             return None
340


~/virtualenv/python3.6.3/lib/python3.6/site-packages/geopy/geocoders/googlev3.py in _check_stat
361         if status == 'OVER_QUERY_LIMIT':
362             raise GeocoderQuotaExceeded(
--> 363                 'The given key has gone over the requests limit in the 24'
364                 ' hour period or has submitted too many requests in too'
365                 ' short a period of time.'


GeocoderQuotaExceeded: The given key has gone over the requests limit in the 24 hour period or
```

```
In [24]: def show_green_in_png(data):
             pixels = img.imread(BytesIO(data)) # Get our PNG image as rows of pixels
             green = is_green(pixels)

             out = green[:,:,np.newaxis] * np.array([0.,1.,0.])[np.newaxis, np.newaxis, :]
             # NOTE: we multiply it for a float array so `img.imsave` works
```

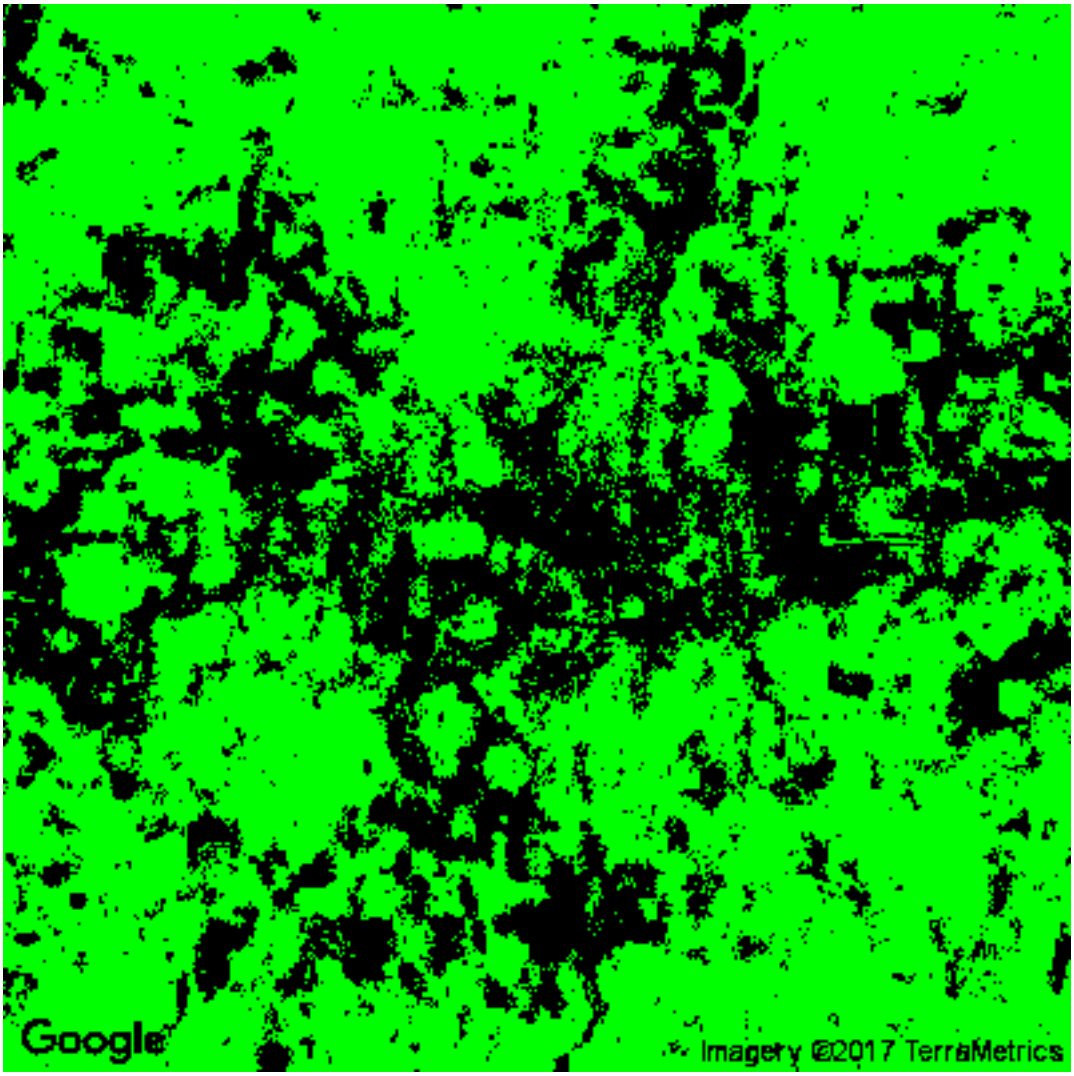
```
        buffer = BytesIO()
        result = img.imsave(buffer, out, format='png')
        return buffer.getvalue()
```

In [25]:
```
display.Image(
    map_at(*london_location)
)
```

Out[25]:

In [26]:
```
display.Image(
    show_green_in_png(
        map_at(
            *london_location)))
```

Out[26]:




```
In [27]: for location in location_sequence(geolocate("London"),
                                           geolocate("Birmingham"),4):
             display.display(display.Image(
                     map_at(*location)))
```

Google
Imagery ©2017 TerraMetrics

Google
Imagery ©2017 TerraMetrics

Google
Imagery ©2017 TerraMetrics

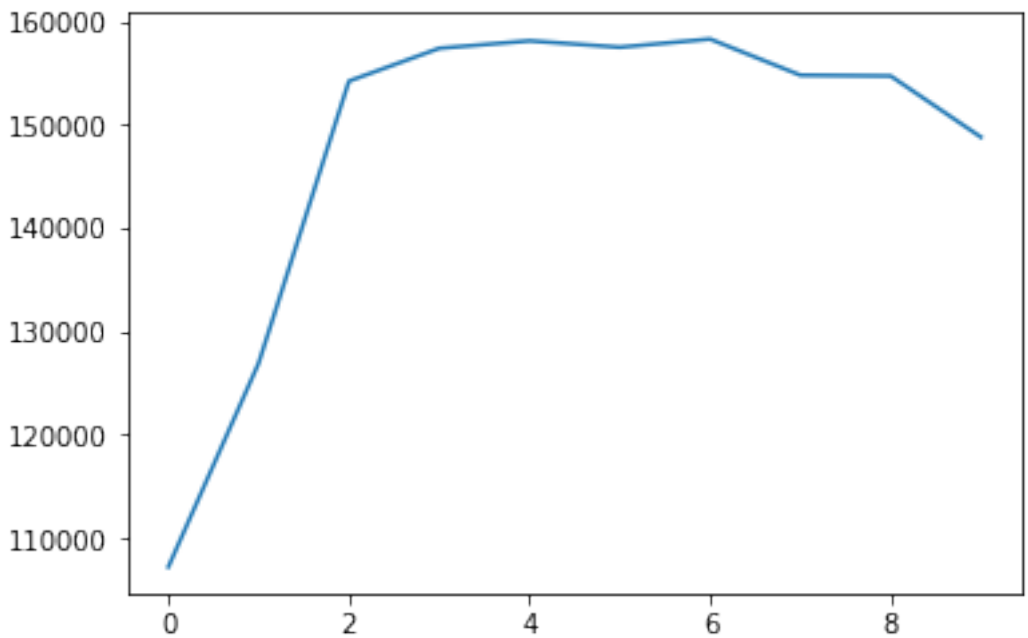
```
In [28]: [count_green_in_png(map_at(*location))
                for location in location_sequence(geolocate("London"),
                                                  geolocate("Birmingham"),
                                                  10)]
```

```
Out[28]: [107222,
          126944,
          154240,
          157417,
          158139,
          157518,
          158311,
          154812,
          154740,
          148817]
```

```
In [29]: import matplotlib.pyplot as plt
```

```
In [30]: plt.plot([count_green_in_png(map_at(*location))
                    for location in location_sequence(geolocate("London"),
                                                      geolocate("Birmingham"),
                                                      10)])
```

```
Out[30]: [<matplotlib.lines.Line2D at 0x2b7b13d8e6a0>]
```




```
In [31]: def green_between(start, end,steps):
             return [count_green_in_png(map_at(*location))
                     for location in location_sequence(
                                     geolocate(start),
                                     geolocate(end),
                                     steps)]
```

```
In [32]: plt.plot(green_between('Rio de Janeiro', 'Buenos Aires', 20))
```

```
---------------------------------------------------------------------------

GeocoderQuotaExceeded                     Traceback (most recent call last)

<ipython-input-32-83d5895079a6> in <module>()
----> 1 plt.plot(green_between('Rio de Janeiro', 'Buenos Aires', 20))


<ipython-input-31-3b4d5cf89054> in green_between(start, end, steps)
      3             for location in location_sequence(
      4                             geolocate(start),
----> 5                             geolocate(end),
      6                             steps)]
```

```
    <ipython-input-5-2394ca7f2ca5> in geolocate(place)
      1 def geolocate(place):
----> 2   return geocoder.geocode(place, exactly_one = False)[0][1]


    ~/virtualenv/python3.6.3/lib/python3.6/site-packages/geopy/geocoders/googlev3.py in geocode(sel
    215         logger.debug("%s.geocode: %s", self.__class__.__name__, url)
    216         return self._parse_json(
--> 217             self._call_geocoder(url, timeout=timeout), exactly_one
    218         )
    219


    ~/virtualenv/python3.6.3/lib/python3.6/site-packages/geopy/geocoders/googlev3.py in _parse_json
    336         places = page.get('results', [])
    337         if not len(places):
--> 338             self._check_status(page.get('status'))
    339             return None
    340


    ~/virtualenv/python3.6.3/lib/python3.6/site-packages/geopy/geocoders/googlev3.py in _check_stat
    361         if status == 'OVER_QUERY_LIMIT':
    362             raise GeocoderQuotaExceeded(
--> 363                 'The given key has gone over the requests limit in the 24'
    364                 ' hour period or has submitted too many requests in too'
    365                 ' short a period of time.'


    GeocoderQuotaExceeded: The given key has gone over the requests limit in the 24 hour period or
```

```
In [1]: 2 * 3

Out[1]: 6

In [2]: six = 2 * 3

In [3]: print(six)

6


In [4]: print(seven)
```

```
    ---------------------------------------------------------------------------

    NameError                                 Traceback (most recent call last)

    <ipython-input-4-25c0309421cb> in <module>()
----> 1 print(seven)
```

```
        NameError: name 'seven' is not defined
```

```
In [5]: nothing = None
In [6]: print(nothing)
None
In [7]: print(5 * six)
30
In [8]: scary = six * six * six
In [9]: print(scary)
216
In [10]: scary = 25
In [11]: print(scary)
25
In [12]: name = "James"
In [13]: nom = name
In [14]: print(nom)
James
In [15]: print(name)
James
In [16]: nom = "Hetherington"
In [17]: print(name)
James
In [18]: print(nom)
Hetherington
In [19]: name = "Jim"
In [20]: type(name)
Out[20]: str
```

```
In [21]: z = 3 + 1j

In [22]: type(z)

Out[22]: complex

In [23]: z.real

Out[23]: 3.0

In [24]: z.imag

Out[24]: 1.0

In [25]: x = "Banana"

In [26]: type(x)

Out[26]: str

In [27]: x.real
```

```
---------------------------------------------------------------------------

AttributeError                            Traceback (most recent call last)

<ipython-input-27-afe486272bd1> in <module>()
----> 1 x.real


AttributeError: 'str' object has no attribute 'real'
```

```
In [28]: z.wrong
```

```
---------------------------------------------------------------------------

AttributeError                            Traceback (most recent call last)

<ipython-input-28-0cc5a8ef8f99> in <module>()
----> 1 z.wrong


AttributeError: 'complex' object has no attribute 'wrong'
```

```
In [29]: second_complex_number = 5 - 6j
         print("Gets to here")
         print(second_complex_number.wrong)
         print("Didn't get to here")
```

```
Gets to here
```

```
---------------------------------------------------------------------------

AttributeError                            Traceback (most recent call last)

<ipython-input-29-60924ebc7ac0> in <module>()
  1 second_complex_number = 5 - 6j
  2 print("Gets to here")
----> 3 print(second_complex_number.wrong)
  4 print("Didn't get to here")


AttributeError: 'complex' object has no attribute 'wrong'
```

```
In [30]: number = 0

In [31]: print(number)
```

```
0
```

```
In [32]: number = number + 1

In [33]: print(number)
```

```
1
```

```
In [1]: len("pneumonoultramicroscopicsilicovolcanoconiosis")

Out[1]: 45

In [2]: len([3,5,15])

Out[2]: 3

In [3]: sorted("Python")

Out[3]: ['P', 'h', 'n', 'o', 't', 'y']

In [4]: name = 'Jim'

In [5]: len(name) * 8

Out[5]: 24

In [6]: x = len('Mike')
        y = len('Bob')
        z = x + y

In [7]: print(z)
```

```
7
```

```
In [8]: "shout".upper()

Out[8]: 'SHOUT'
```

```
In [9]: "WhiSpER".lower()

Out[9]: 'whisper'

In [10]: x = 5
         x.upper()


        ---------------------------------------------------------------------------

        AttributeError                            Traceback (most recent call last)

        <ipython-input-10-2f7864fa034f> in <module>()
          1 x = 5
    ----> 2 x.upper()


        AttributeError: 'int' object has no attribute 'upper'


In [11]: x = 5
         x.wrong


        ---------------------------------------------------------------------------

        AttributeError                            Traceback (most recent call last)

        <ipython-input-11-8f1e8a62ff4e> in <module>()
          1 x = 5
    ----> 2 x.wrong


        AttributeError: 'int' object has no attribute 'wrong'


In [12]: z = 1 + 5j

In [13]: z.real

Out[13]: 1.0

In [14]: z.conjugate()

Out[14]: (1-5j)

In [15]: z.conjugate

Out[15]: <function complex.conjugate>

In [16]: type(z.conjugate)

Out[16]: builtin_function_or_method

In [17]: somefunc = z.conjugate

In [18]: somefunc()
```

```
Out[18]: (1-5j)

In [19]: magic = sorted

In [20]: type(magic)

Out[20]: builtin_function_or_method

In [21]: magic(["Technology", "Advanced"])

Out[21]: ['Advanced', 'Technology']

In [22]: def double(data):
             return data * 2

In [23]: double(5)

Out[23]: 10

In [24]: double

Out[24]: <function __main__.double>

In [25]: timestwo = double

In [26]: timestwo(8)

Out[26]: 16

In [27]: timestwo(5, 6, 7)

         ---------------------------------------------------------------------------

         TypeError                                 Traceback (most recent call last)

         <ipython-input-27-576707f347a8> in <module>()
     ----> 1 timestwo(5, 6, 7)


         TypeError: double() takes 1 positional argument but 3 were given

In [28]: def timesten(input):
             return 5 * double(input)

In [29]: timesten(4)

Out[29]: 40

In [30]: help(sorted)

Help on built-in function sorted in module builtins:

sorted(iterable, /, *, key=None, reverse=False)
    Return a new list containing all items from the iterable in ascending order.

    A custom key function can be supplied to customize the sort order, and the
    reverse flag can be set to request the result in descending order.
```

```
In [31]: help(z.conjugate)
```

```
Help on built-in function conjugate:

conjugate(...) method of builtins.complex instance
    complex.conjugate() -> complex

    Return the complex conjugate of its argument. (3-4j).conjugate() == 3+4j.
```

```
In [32]: dir("Hexxo")
```

```
Out[32]: ['__add__',
          '__class__',
          '__contains__',
          '__delattr__',
          '__dir__',
          '__doc__',
          '__eq__',
          '__format__',
          '__ge__',
          '__getattribute__',
          '__getitem__',
          '__getnewargs__',
          '__gt__',
          '__hash__',
          '__init__',
          '__init_subclass__',
          '__iter__',
          '__le__',
          '__len__',
          '__lt__',
          '__mod__',
          '__mul__',
          '__ne__',
          '__new__',
          '__reduce__',
          '__reduce_ex__',
          '__repr__',
          '__rmod__',
          '__rmul__',
          '__setattr__',
          '__sizeof__',
          '__str__',
          '__subclasshook__',
          'capitalize',
          'casefold',
          'center',
          'count',
          'encode',
          'endswith',
          'expandtabs',
          'find',
          'format',
```

```
          'format_map',
          'index',
          'isalnum',
          'isalpha',
          'isdecimal',
          'isdigit',
          'isidentifier',
          'islower',
          'isnumeric',
          'isprintable',
          'isspace',
          'istitle',
          'isupper',
          'join',
          'ljust',
          'lower',
          'lstrip',
          'maketrans',
          'partition',
          'replace',
          'rfind',
          'rindex',
          'rjust',
          'rpartition',
          'rsplit',
          'rstrip',
          'split',
          'splitlines',
          'startswith',
          'strip',
          'swapcase',
          'title',
          'translate',
          'upper',
          'zfill']
```

```
In [33]: "Hexxo".replace("x", "l")
```

```
Out[33]: 'Hello'
```

```
In [34]: x = 2 + 3
```

```
In [35]: print(x)
```

```
5
```

```
In [36]: x.__add__(7)
```

```
Out[36]: 12
```

```
In [37]: [2, 3, 4] + [5, 6]
```

```
Out[37]: [2, 3, 4, 5, 6]
```

```
In [38]: 7 - 2
```

```
Out[38]: 5

In [39]: [2, 3, 4] - [5, 6]

         ---------------------------------------------------------------------------

         TypeError                                 Traceback (most recent call last)

         <ipython-input-39-5b64b789ad11> in <module>()
     ----> 1 [2, 3, 4] - [5, 6]


         TypeError: unsupported operand type(s) for -: 'list' and 'list'

In [40]: [2, 3, 4] + 5

         ---------------------------------------------------------------------------

         TypeError                                 Traceback (most recent call last)

         <ipython-input-40-67b01a5c24ab> in <module>()
     ----> 1 [2, 3, 4] + 5


         TypeError: can only concatenate list (not "int") to list

In [41]: print(2 + 3 * 4)

14

In [42]: print((2 + 3) * 4)

20

In [1]: type(5)

Out[1]: int

In [2]: one = 1
        ten = 10
        one_float = 1.0
        ten_float = 10.

In [3]: tenth = one_float / ten_float

In [4]: tenth

Out[4]: 0.1

In [5]: one // ten

Out[5]: 0
```

```
In [6]: 2 + 3

Out[6]: 5

In [7]: "Hello " + "World"

Out[7]: 'Hello World'

In [8]: print(one // ten, one / ten)

0 0.1

In [9]: print(type(one // ten), type(one / tenth))

<class 'int'> <class 'float'>

In [10]: 10 // 3

Out[10]: 3

In [11]: 10 / 3

Out[11]: 3.3333333333333335

In [12]: 10 // 3.0

Out[12]: 3.0

In [13]: str(5)

Out[13]: '5'

In [14]: 10 / float(3)

Out[14]: 3.3333333333333335

In [15]: x = 5

In [16]: str(x)

Out[16]: '5'

In [17]: x = "5.2"

         int(float(x))

Out[17]: 5

In [18]: given = "James"
         family = "Hetherington"
         full = given + " " + family

In [19]: print(full.upper())

JAMES HETHERINGTON

In [20]: 2 * 4
```

```
Out[20]: 8

In [21]: ten

Out[21]: 10

In [22]: one

Out[22]: 1

In [23]: type(ten)

Out[23]: int

In [24]: print(ten + one)

11

In [25]: print(float(str(ten) + str(one)))

101.0

In [26]: 5

Out[26]: 5

In [27]: "    Hello  ".strip()

Out[27]: 'Hello'

In [28]: [1, 3, 7]

Out[28]: [1, 3, 7]

In [29]: type([1, 3, 7])

Out[29]: list

In [30]: various_things = [1, 2, "banana", 3.4, [1,2] ]

In [31]: one = 1
         two = 2
         three = 3

In [32]: my_new_list = [one, two, three]

In [33]: middle_value_in_list = my_new_list[1]

In [34]: middle_value_in_list

Out[34]: 2

In [35]: [1,2,3][1]

Out[35]: 2

In [36]: various_things[2]

Out[36]: 'banana'
```

```
In [37]: index = 2
         various_things[index]

Out[37]: 'banana'

In [38]: count_to_five = list(range(5))
         print(count_to_five)

[0, 1, 2, 3, 4]


In [39]: name = ["James", "Philip", "John", "Hetherington"]
         print(" -> ".join(name))

James -> Philip -> John -> Hetherington


In [40]: "Ernst Stavro Blofeld".split("o")

Out[40]: ['Ernst Stavr', ' Bl', 'feld']

In [41]: name.append("Jr")
         print(name)

['James', 'Philip', 'John', 'Hetherington', 'Jr']


In [42]: name.extend(["the", "third"])
         print(name)

['James', 'Philip', 'John', 'Hetherington', 'Jr', 'the', 'third']


In [43]: print(count_to_five[1])
         print("James"[2])

1
m


In [44]: print(count_to_five[1:3])
         print("Hello World"[4:8])

[1, 2]
o Wo


In [45]: print(len(various_things))
         print(len("Python"))

5
6


In [46]: len([[1, 2], 4])

Out[46]: 2
```

```
In [47]: print("John" in name)
         print(9 in count_to_five)
```

```
True
False
```

```
In [48]: mylist = ['Goodbye', 'Cruel']
         a, b = mylist
         print(a)
```

```
Goodbye
```

```
In [49]: a = mylist[0]
         b = mylist[1]
```

```
In [1]: 'Dog' in ['Cat', 'Dog', 'Horse']

Out[1]: True

In [2]: 'Bird' in ['Cat', 'Dog', 'Horse']

Out[2]: False

In [3]: 2 in range(5)

Out[3]: True

In [4]: 99 in range(5)

Out[4]: False

In [5]: 'a' in 'cat'

Out[5]: True

In [6]: name =  "James Philip John Hetherington".split(" ")
        print(name)
```

```
['James', 'Philip', 'John', 'Hetherington']
```

```
In [7]: name[0] = "Dr"
        name[1:3] = ["Griffiths-"]
        name.append("PhD")

        print(" ".join(name))
```

```
Dr Griffiths- Hetherington PhD
```

```
In [8]: my_tuple = ("Hello", "World")

In [9]: my_tuple

Out[9]: ('Hello', 'World')

In [10]: my_tuple[0]="Goodbye"
```

```
---------------------------------------------------------------------------

TypeError                                 Traceback (most recent call last)

<ipython-input-10-2923ed76d341> in <module>()
----> 1 my_tuple[0]="Goodbye"


TypeError: 'tuple' object does not support item assignment
```

```
In [11]: fish = "Hake"
         fish[0] = 'R'
```

```
---------------------------------------------------------------------------

TypeError                                 Traceback (most recent call last)

<ipython-input-11-7127277fc72e> in <module>()
  1 fish = "Hake"
----> 2 fish[0] = 'R'


TypeError: 'str' object does not support item assignment
```

```
In [12]: fish = "Rake" ## OK!
```

```
In [13]: x = list(range(3))
         print(x)
```

```
[0, 1, 2]
```

```
In [14]: y = x
         print(y)
```

```
[0, 1, 2]
```

```
In [15]: z = x[0:3]
         y[1] = "Gotcha!"
         print(x)
         print(y)
         print(z)
```

```
[0, 'Gotcha!', 2]
[0, 'Gotcha!', 2]
[0, 1, 2]
```

```
In [16]: z[2] = "Really?"
         print(x)
         print(y)
         print(z)
```

```
[0, 'Gotcha!', 2]
[0, 'Gotcha!', 2]
[0, 1, 'Really?']
```

```
In [17]: x = ["What's", "Going", "On?"]
         y = x
         z = x[0:3]

         y[1] = "Gotcha!"
         z[2] = "Really?"
```

```
In [18]: x
```

```
Out[18]: ["What's", 'Gotcha!', 'On?']
```

```
In [19]: x = [['a', 'b'], 'c']
         y = x
         z = x[0:2]

         x[0][1] ='d'
         z[1] ='e'
```

```
In [20]: x
```

```
Out[20]: [['a', 'd'], 'c']
```

```
In [21]: y
```

```
Out[21]: [['a', 'd'], 'c']
```

```
In [22]: z
```

```
Out[22]: [['a', 'd'], 'e']
```

```
In [23]: print([1, 2] == [1, 2])
         print([1, 2] is [1, 2])
```

```
True
False
```

```
In [24]: my3numbers = list(range(3))
         print(my3numbers)
```

```
[0, 1, 2]
```

```
In [25]: [0, 1, 2] == my3numbers
```

```
Out[25]: True
```

```
In [26]: [0, 1, 2] is my3numbers
```

```
Out[26]: False
```

```
In [27]: word = "Hello"
         print("Hello" == word)
         print("Hello" is word)
```

```
True
True
```

```
In [1]: names="Martin Luther King".split(" ")
        names[1]

Out[1]: 'Luther'

In [2]: me = { "name": "James", "age": 39, "Jobs": ["Programmer", "Teacher"] }

In [3]: print(me)

{'name': 'James', 'age': 39, 'Jobs': ['Programmer', 'Teacher']}

In [4]: print(me['Jobs'])

['Programmer', 'Teacher']

In [5]: print(type(me))

<class 'dict'>

In [6]: me.keys()

Out[6]: dict_keys(['name', 'age', 'Jobs'])

In [7]: me.values()

Out[7]: dict_values(['James', 39, ['Programmer', 'Teacher']])

In [8]: 'Jobs' in me

Out[8]: True

In [9]: 'James' in me

Out[9]: False

In [10]: 'James' in me.values()

Out[10]: True

In [11]: good_match = {("Lamb", "Mint"): True, ("Bacon", "Chocolate"): False}

In [12]: illegal = {[1,2]: 3}

        ---------------------------------------------------------------------------

        TypeError                                 Traceback (most recent call last)

        <ipython-input-12-b4b5eed8fa3b> in <module>()
    ----> 1 illegal = {[1,2]: 3}


        TypeError: unhashable type: 'list'
```

```
In [13]: my_dict = {'0': 0, '1':1, '2': 2, '3': 3, '4': 4}
         print(my_dict)
         print(my_dict.values())
```

```
{'0': 0, '1': 1, '2': 2, '3': 3, '4': 4}
dict_values([0, 1, 2, 3, 4])
```

```
In [14]: university = "University College London"
         unique_letters = set(university)
```

```
In [15]: unique_letters
```

```
Out[15]: {' ',
          'C',
          'L',
          'U',
          'd',
          'e',
          'g',
          'i',
          'l',
          'n',
          'o',
          'r',
          's',
          't',
          'v',
          'y'}
```

```
In [16]: print("".join(unique_letters))
```

```
eyU oLgsClnvrdit
```

```
In [17]: "".join(['a', 'b', 'c'])
```

```
Out[17]: 'abc'
```

```
In [18]: alist = [1, 2, 3]
         is_unique = len(set(alist)) == len(alist)
         print(is_unique)
```

```
True
```

```
In [19]: x = {'a':1, 'b':2}
```

```
In [20]: x['a']
```

```
Out[20]: 1
```

```
In [21]: x['fish']
```

```
KeyError                                  Traceback (most recent call last)

<ipython-input-21-1ed14b9eae71> in <module>()
----> 1 x['fish']


KeyError: 'fish'
```

```
In [22]: x.get('a')
```

```
Out[22]: 1
```

```
In [23]: x.get('fish')
```

```
In [24]: x.get('fish', 'tuna') == 'tuna'
```

```
Out[24]: True
```

```
In [1]: UCL = {'City': 'London',
             'Street': 'Gower Street',
             'Postcode': 'WC1E 6BT'}
```

```
In [2]: MyHouse = {
            'City': 'London',
            'Street': 'Waterson Street',
            'Postcode': 'E2 8HH'
        }
```

```
In [3]: addresses = [UCL, MyHouse]
```

```
In [4]: addresses
```

```
Out[4]: [{'City': 'London', 'Postcode': 'WC1E 6BT', 'Street': 'Gower Street'},
         {'City': 'London', 'Postcode': 'E2 8HH', 'Street': 'Waterson Street'}]
```

```
In [5]: UCL['People'] = ['Clare','James', 'Owain']
```

```
In [6]: MyHouse['People'] = ['Sue', 'James']
```

```
In [7]: addresses
```

```
Out[7]: [{'City': 'London',
          'People': ['Clare', 'James', 'Owain'],
          'Postcode': 'WC1E 6BT',
          'Street': 'Gower Street'},
         {'City': 'London',
          'People': ['Sue', 'James'],
          'Postcode': 'E2 8HH',
          'Street': 'Waterson Street'}]
```

```
In [8]: UCL['Residential'] = False
```

```
In [9]: leaders = [place['People'][0] for place in addresses]
        print(leaders)
```

```
['Clare', 'Sue']
```

```
In [10]: cities = [{'name': "London", "capacity": 8, "residents": ["Me","Sue"]},
          {'name': "Edinburgh", "capacity": 1, "residents": ["Dave"]},
          {'name': "Cardiff", "capacity": 1, "residents": []}]
```

```
In [11]: len(cities[2]['residents'])
```

```
Out[11]: 0
```

```
In [1]: house = {
            'living' : {
                'exits': {
                    'north' : 'kitchen',
                    'outside' : 'garden',
                    'upstairs' : 'bedroom'
                },
                'people' : ['James'],
                'capacity' : 2
            },
            'kitchen' : {
                'exits': {
                    'south' : 'living'
                },
                'people' : [],
                'capacity' : 1
            },
            'garden' : {
                'exits': {
                    'inside' : 'living'
                },
                'people' : ['Sue'],
                'capacity' : 3
            },
            'bedroom' : {
                'exits': {
                    'downstairs' : 'living',
                    'jump' : 'garden'
                },
                'people' : [],
                'capacity' : 1
            }
        }
```

```
In [2]: people_so_far = 0

        for room_name in house:
            people_so_far = people_so_far + len(house[room_name]['people'])

        print(people_so_far)
```

```
2
```

```
In [1]: x = -3
        if x < 0:
            print(x, "is negative")
            print("This is controlled")
        print("Always run this")
```

```
-3 is negative
This is controlled
Always run this
```

```
In [2]: x = -3
        if x < 0:
            print("x is negative")
        else:
            print("x is positive")
```

```
x is negative
```

```
In [3]: x = 5
        if x < 0:
            print("x is negative")
        elif x == 0:
            print("x is zero")
        else:
            print("x is positive")
```

```
x is positive
```

```
In [4]: choice ='dlgkhdglkhgkjhdkjgh'

        if choice == 'high':
            print(1)
        elif choice == 'medium':
            print(2)
        else:
            print(3)
```

```
3
```

```
In [5]: 1 > 2

Out[5]: False

In [6]: "UCL" > "King's"

Out[6]: True

In [7]: True == "True"

Out[7]: False

In [8]: '1' < 2
```

```
        ---------------------------------------------------------------------------

        TypeError                                 Traceback (most recent call last)

        <ipython-input-8-2ae56e567bff> in <module>()
```